

ITEL 5 – Advanced Information Management

1. Transaction Model

Suppose that accounts A and B are \$1000 and \$2000, respectively. Account A will transfer 50 to account B. Write the SQL statement.

```
BEGIN TRANSACTION;

UPDATE Accounts
SET balance = balance - 50
WHERE account_id = 'A';

UPDATE Accounts
SET balance = balance + 50
WHERE account_id = 'B';

COMMIT;
```

If any error occurs (e.g., insufficient funds), you can use ROLLBACK to undo all changes:

```
ROLLBACK;
```

2. Transaction Control and Recovery in SQL

Learning Objectives

By the end of this lecture, students should be able to:

- Create a database and tables for transaction management.
- Execute transactions with COMMIT, ROLLBACK, and ABORT.
- Understand how logs record changes for recovery.
- Apply compensating transactions to reverse committed changes.
- Differentiate between restartable and killable transactions.
- Apply SQL commands for rollback, restart, and termination.
- Understand how logs track transaction states.

Transaction Atomicity and Durability

Step 1: Create the Database and Tables

```
CREATE DATABASE BankDB;
USE BankDB;

CREATE TABLE Accounts (
    account_id CHAR(1) PRIMARY KEY,
    balance DECIMAL(10,2)
);

CREATE TABLE TransactionLog (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
```

```

transaction_id VARCHAR(10),
account_id CHAR(1),
old_value DECIMAL(10,2),
new_value DECIMAL(10,2),
action VARCHAR(20),
log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Step 2: Insert Initial Values

```

INSERT INTO Accounts VALUES ('A', 1000.00);
INSERT INTO Accounts VALUES ('B', 2000.00);

```

Step 3: Begin Transaction (Transfer \$50 from A to B)

```

START TRANSACTION;

-- Log before modification
INSERT INTO TransactionLog (transaction_id, account_id, old_value, new_value,
action)
VALUES ('T1', 'A', 1000.00, 950.00, 'DEBIT');

INSERT INTO TransactionLog (transaction_id, account_id, old_value, new_value,
action)
VALUES ('T1', 'B', 2000.00, 2050.00, 'CREDIT');

-- Perform updates
UPDATE Accounts SET balance = balance - 50 WHERE account_id = 'A';
UPDATE Accounts SET balance = balance + 50 WHERE account_id = 'B';

```

Step 4: Aborted Transaction (Simulated Failure)

Suppose an error occurs during execution:

```

-- Simulate failure
ROLLBACK;

```

Result:

- All changes are undone.
- The database returns to its original state.
- Log entries remain for recovery analysis.

Step 5: Successful Transaction (Commit)

If no error occurs:

```

COMMIT;

```

Result:

- Changes are permanently saved.
- The transaction is marked as committed in the log.

Step 6: Compensating Transaction

If a committed transaction needs to be reversed manually:

```
START TRANSACTION;
```

```
-- Log compensating actions
```

```
INSERT INTO TransactionLog (transaction_id, account_id, old_value, new_value,  
action)
```

```
VALUES ('CT1', 'A', 950.00, 1000.00, 'COMPENSATE');
```

```
INSERT INTO TransactionLog (transaction_id, account_id, old_value, new_value,  
action)
```

```
VALUES ('CT1', 'B', 2050.00, 2000.00, 'COMPENSATE');
```

```
-- Reverse the previous transaction
```

```
UPDATE Accounts SET balance = balance + 50 WHERE account_id = 'A';
```

```
UPDATE Accounts SET balance = balance - 50 WHERE account_id = 'B';
```

```
COMMIT;
```

Key Concepts Illustrated

Concept	Description
Aborted Transaction	Fails before completion; changes undone.
Rollback	Restores database to previous consistent state.
Log	Records old and new values for recovery.
Commit	Confirms successful completion; changes persist.
Compensating Transaction	Manually reverses effects of a committed transaction.

Restart Transaction (New Transaction After Recovery)

After recovery, the system restarts the transaction as a new one

```
START TRANSACTION;
```

```
-- Log restart
```

```
INSERT INTO TransactionLog (transaction_id, account_id, old_value, new_value,  
action)
```

```
VALUES ('T2R', 'A', 1000.00, 950.00, 'RESTART-DEBIT');
```

```
INSERT INTO TransactionLog (transaction_id, account_id, old_value, new_value,  
action)
```

```
VALUES ('T2R', 'B', 2000.00, 2050.00, 'RESTART-CREDIT');
```

```
-- Reapply updates
```

```
UPDATE Accounts SET balance =
```

Outcome:

- Transaction successfully restarted and committed.
- Database returns to a consistent state.

Kill Transaction (Logical or Input Error)

If the abort was due to bad input or logical error, the transaction must be terminated:

START TRANSACTION;

-- Example of bad input

UPDATE Accounts SET balance = balance - 5000 WHERE account_id = 'A'; --
insufficient funds

-- Detect logical error

ROLLBACK;

-- Log termination

INSERT INTO TransactionLog (transaction_id, account_id, old_value, new_value,
action)

VALUES ('T3', 'A', 1000.00, 1000.00, 'KILLED-LOGICAL-ERROR');

Explanation:

- The transaction is killed, not restarted.
- Requires application-level correction (e.g., fix logic or input validation).

Scenario	SQL Action	Result
Hardware/Software Error	ROLLBACK → Restart as new transaction	Transaction restarted (T2R)
Logical/Input Error	ROLLBACK → Kill transaction	Transaction terminated (T3)

Key Takeaways

- Restarted transactions are treated as new transactions after recovery.
- Killed transactions require program correction, not database recovery.
- Logging ensures traceability of all transaction states.
- Atomicity and durability are preserved through rollback and commit control.

Transaction Lifecycle Flow

Made with AI

